Project Documentation & Architecture Overview
Club 33 Canteen

> **Scope note:** This describes the currently registered Flask application and the templates and assets wired into it. Some numbered templates and similarly named scripts appear to be alternate or legacy versions. The Hybrid Partial Render pattern is implemented in order history, but is **not a global rendering rule** across the application.

1. Project Overview & Core Stack

Club 33 is a canteen ordering application. Customers browse a menu, build a session-based cart, submit orders, and view order confirmations. Staff use an admin area to manage the menu and categories, process orders, and view analytics and order history.

Layer	Technology and role
Backend	Python and Flask, using an application factory in `app/__init__.py`.
Persistence	Flask-SQLAlchemy; SQLite by default, configurable through `DATABASE_URL` in `config.py`.
Templates	Jinja2, via Flask templates in `app/templates/`.
UI framework	Bootstrap 5.3.3 and Bootstrap Icons, loaded from CDNs in the base templates.
Styling	SCSS compiled with Libsass; modular source is in `app/static/scss/`.
Partial updates	HTMX 1.9.10, loaded in the admin base template; used chiefly for order-history filtering and pagination.
Browser logic	JavaScript modules and scripts using `fetch` for JSON APIs.
Other dependencies	Flask-Migrate, Flask-Login, Flask-WTF, `python-dotenv`; menu image processing imports Pillow.

The declared Python packages are listed in `requirements.txt`. The application factory initializes SQLAlchemy and Flask-Migrate and calls `db.create_all()` on startup. The default database path is `instance/club33.db`.

2. Database Schema & Data Flow

All ORM models are defined in `app/models.py`.

Models

Model / table	Columns
`Category` `categories`	`id` — Integer, primary key. `name` — String(64), required, unique. `display_order` — Integer, required, default `0`.
`MenuItem` `menu_items`	`id` — Integer, primary key. `name` — String(100), required. `description` — String(255), default `""`. `price` — Float, required. `category_id` — Integer, nullable foreign key to `categories.id`, with `ON DELETE SET NULL`. `image_filename` — String(120), default `"placeholder.jpg"`. `is_available` — Boolean, required, default `True`.
`Order` `orders`	`id` — Integer, primary key. `order_number` — String(16), required, unique, indexed. `customer_name` — String(100), required. `phone` — String(20), required. `status` — String(20), required, default `"pending"`. `total_price` — Float, required, default `0.0`. `created_at` — timezone-aware DateTime column, required, default Singapore time (UTC+8). `ready_at`, `completed_at`, `cancelled_at` — nullable timezone-aware DateTime columns.
`OrderItem` `order_items`	`id` — Integer, primary key. `order_id` — required Integer foreign key to `orders.id`. `menu_item_id` — required Integer foreign key to `menu_items.id`. `quantity` — Integer, required, default `1`. `item_name` — String(100), required; name snapshot at purchase. `unit_price` — Float, required; price snapshot at purchase.
`User` `users`	`id` — Integer, primary key. `username` — String(50), required, unique, indexed. `email` — String(120), required, unique, indexed. `password_hash` — String(256), required. `role` — String(20), required, default `"admin"`. `created_at` — DateTime, default UTC time; the column is not declared with `timezone=True`.

Relationships and order flow

- `Category.items` relates categories to menu items; `MenuItem.category` is the corresponding back-reference. A menu item may have no category.
- `Order.items` relates an order to its line items and configures `cascade="all, delete-orphan"`.
- `OrderItem.menu_item` relates each line to a menu item. `OrderItem.subtotal` is calculated as `unit_price * quantity`.
- A customer's cart is stored in the Flask session as item IDs and quantities. Cart endpoints look up current menu data to calculate totals.
- Order creation reads the session cart, validates the customer name and phone, creates an order and its line items, stores the item name and unit-price snapshots, commits, then clears the cart. The endpoint returns JSON with a confirmation URL.
- Confirmation pages are rendered by the main blueprint. Order lookups and creation are implemented primarily in `app/api/order.py`, with HTML views in `app/main/routes.py`.

Data and contract caveats

- `Order` defines `pending`, `confirmed`, `ready`, `completed`, and `cancelled`. Some order-management JavaScript and analytics code instead use `collected`; the status-update endpoint does not accept `collected`. Treat status names as a cross-layer contract that needs verification.
- Monetary fields use `Float`, not a fixed-precision decimal type.

- Singapore time is used for order timestamps and analytics, while some filtering code works with naive `datetime` values. `User.created_at` uses UTC and a DateTime column without `timezone=True`.

3. Directory Architecture

- `app/` — Flask package.
  - `__init__.py` — application factory, extension setup, and blueprint registration.
  - `extensions.py` — shared SQLAlchemy and Flask-Migrate instances.
  - `models.py` — ORM schema.
  - `admin/` — admin HTML routes.
  - `api/` — JSON-oriented menu, category, cart, order, analytics, and authentication endpoints.
  - `main/` — public HTML pages and order tracking/confirmation views.
  - `services/` — query and order helper logic, including `order_service.py`.
  - `utils/` — shared utilities, including the admin access decorator.
  - `templates/` — Jinja templates:
    - `admin/` — analytics, history, menu management, and order-management pages.
    - `main/` and `menu/` — public landing and menu views.
    - `partials/` — reusable HTML fragments, including history content, pagination, filters, and toasts.
    - `modals/` — add/edit forms and order/login modals.
    - `order_confirmation/`, `track_order/` — order lifecycle views.
    - Base layouts include `base.html`, `base2.html`, `base3.html`, `admin_base.html`, and `admin_base2.html`. Numbered variants also exist for several pages; the routes currently render the unnumbered templates.
  - `static/` — frontend assets:
    - `css/` — generated or alternate CSS files.
    - `scss/` — Sass entry point and `abstracts/`, `base/`, `components/`, `layouts/`, and `utilities/` modules.
    - `js/` — page scripts plus `api/`, `features/`, `pages/`, and `ui/` modules.
    - `img/` and `uploads/` — bundled images and uploaded menu images.
- At the repository root, `run.py` starts the app, `config.py` configures it, and `README.md` provides basic setup instructions. Other root scripts and structure notes include `seeddb.py`, `cleardb.py`, and `test.py`.

The local virtual environment and SQLite database are runtime/development artifacts rather than application modules.

4. Backend Routing & Blueprints

The application factory registers the public, admin, and API blueprints in `app/__init__.py`.

HTML-rendering views

Blueprint	Route	Purpose
`main_bp`	`GET /`	Public landing page; loads categories and menu items.

Blueprint	Route	Purpose
`main_bp`	`GET /menu`	Menu page.
`main_bp`	`GET /confirmation/<order_number>`	Order confirmation view.
`main_bp`	`GET, POST /track_order`	Tracking form and submission/redirect flow.
`admin_bp`	`GET /admin/`	Analytics dashboard page.
`admin_bp`	`GET /admin/manage-menu`	Menu/category management page.
`admin_bp`	`GET /admin/orders`	Order-management page.
`admin_bp`	`GET /admin/history`	Full order-history page or HTMX fragment response.

Admin page routes are protected by a `before_request` check in `app/admin/routes.py`. The shared `admin_required` decorator also checks authentication and the admin role. Authentication uses Flask-Login; the login API is separate from the admin page blueprint.

JSON API endpoints

The `api_bp` blueprint is mounted at `/api`; its child blueprints provide these route groups:

Route group	Primary endpoints	Purpose
Menu	`GET, POST /api/menu/`; `GET, DELETE /api/menu/<item_id>`; `POST /api/menu/update/<item_id>`; `POST /api/menu/toggle/<item_id>/<checked>`	Read, create, update, delete, and toggle menu items.
Categories	`GET /api/categories/`; `POST /add`, `/delete/<category_id>`, `/reorder`, `/update/<category_id>`	Read and manage menu categories.
Cart	`GET /api/cart/get_cart`; `POST /add_to_cart`, `/clear`, `/update_quantity`	Maintain and summarize the session cart.
Orders	`GET, POST /api/order/`; `GET /active`, `/<order_number>`; `PUT /<order_id>/status`, `/complete`, `/cancel`	Create orders, retrieve details, and change status.
Analytics	`GET /api/analytics/`, with a `range` query parameter	Return dashboard aggregates and chart datasets.
Authentication	`POST /api/auth/login`; `GET` or `POST /api/auth/logout`	Login/logout and session handling.

Most API handlers return JSON. Admin menu and order screens use JavaScript to call these endpoints; admin page authorization is applied by the admin blueprint, so API endpoints should not be assumed to inherit that same blueprint guard.

5. Frontend & HTMX Patterns

Hybrid Partial Render: implemented on order history

1. `admin/history.html` issues `hx-get` requests to `/admin/history`, targeting `#historyResults`. It uses `hx-push-url="true"` so the browser URL reflects the active query.
2. The search input requests updates after `input changed delay:300ms` and includes `#filterForm`, so filter controls accompany the search request.
3. The `/admin/history` route reads `request.args`, filters and paginates results, and checks for the `HX-Request` header. HTMX requests receive only `partials/_history_content.html`; normal navigation receives the full page.
4. The fragment uses `hx-swap-oob="true"` on the filter badge container, updating it outside the primary `#historyResults` target.
5. `partials/_pagination.html` defines a `page_url(p)` Jinja macro that carries the current search, status, date, and price parameters into pagination links.

In this history workflow, the query string is the rendered state for search, filters, and pagination: the service reads those arguments, and the template uses `request.args` to repopulate form values. The shared filtering helper is `get_filtered_orders_query` in `app/services/order_service.py`.

Layout stability

The history fragment uses a fixed-layout table and a `<colgroup>` with explicit column widths. Its surrounding results container has a minimum height, and the pagination is outside the table in a flex layout. These choices reduce table-column movement and keep pagination positioned consistently during fragment replacement. Bootstrap flex utilities are also used throughout the layouts.

Scope and limits of the pattern

- This is **not strict, app-wide HTMX rendering**.
- The HTMX attributes and `HX-Request` response branch are concentrated in order history and its partials.
- Menu, cart, order, and analytics interactions primarily use JavaScript `fetch` calls to JSON endpoints.
- The admin menu route reads a `search` argument, but its filtering code is commented out and it loads all menu items. Therefore, the history query-string state pattern should not be generalized to menu search or every page.
- The project uses includes for shared fragments and has a pagination macro; it does not appear to have a shared macro library for all components.

6. Known Standards & Conventions

- **Application structure:** Flask application factory plus extension instances in `app/extensions.py`; blueprints separate public views, admin views, and APIs.
- **Pagination:** Order history uses 10 orders per page and a sliding page-number window from `calculate_pagination_window` in `app/admin/routes.py`.
- **Filtering:** `get_filtered_orders_query` in `app/services/order_service.py` supports text search across order number/name/phone, multiple status values, date presets or custom dates, and minimum/maximum totals; it sorts newest first.

- **Order constants:** Prefer the `Order.STATUS_*` constants in `app/models.py` over arbitrary status strings. Current frontend/API status vocabulary is inconsistent, as noted above.
- **SCSS organization:** `main.scss` imports the active variable file and modular base, component, layout, and utility styles. `abstracts/_variables.scss` defines color, semantic, typography, and UI tokens.
- **Bootstrap integration:** Bootstrap CSS is loaded from a CDN, followed by project styles. This is a custom SCSS layer, not a Sass build of Bootstrap itself; project Sass variables do not automatically recompile the CDN stylesheet.
- **Build wiring to verify:** `run.py` compiles SCSS from `app/static/scss` into `app/static/css` in debug mode, while `base.html` links to `static/scss/main.css`. Confirm the intended stylesheet output and link before relying on startup compilation to update the served styles.
- **Image uploads:** Menu image handling is in `app/api/menu.py`; it uses Pillow and stores files under the configured upload directory. Pillow is used in code but is not listed in `requirements.txt`.